

```

1 import sys
2
3 from PySide6.QtWidgets import QApplication
4
5 from SERVICIOS.persona import PersonaServicio
6
7 app = QApplication()
8 vtn_principal = PersonaServicio()
9 vtn_principal.show()
10 sys.exit(app.exec())
11

```

Run console output:

```

"C:\PROGRAMACION\QT DESIGNER\PROYECTOSMB\.venv\Scripts\python.exe" "C:\PROGRAMACION\QT DESIGNER\PROYECTOSMB\main.py"
Persona: Brigitte Solis, Cédula: 0987654312, Sexo: femenino, Email: brigitte213@mdi.com
Process finished with exit code 0

```

```

1 import re
2
3 class Persona:
4     """
5     Clase que permite crear objetos de tipo persona
6     """
7
8     def __init__(self, cedula: str, nombre: str, apellido: str, sexo: str, email: str = None):
9         self.cedula = cedula
10        self.nombre = nombre
11        self.apellido = apellido
12        self.sexo = sexo
13        self.email = email
14
15        # Cedula
16        @property
17        def cedula(self):
18            return self._cedula
19
20        @cedula.setter
21        def cedula(self, valor):
22            if not valor.isdigit():
23                raise ValueError("La cédula debe contener solo números")
24            self._cedula = valor
25
26        @property
27        def nombre(self):

```

Run console output:

```

"C:\PROGRAMACION\QT DESIGNER\PROYECTOSMB\.venv\Scripts\python.exe" "C:\PROGRAMACION\QT DESIGNER\PROYECTOSMB\main.py"

```

The screenshot shows a code editor with the file `PERSONA\persona.py` open. The code defines a `Persona` class with the following methods:

```

class Persona:
    @property
    def nombre(self):
        return self._nombre

    @nombre.setter
    def nombre(self, valor):
        if not valor.isalpha():
            raise ValueError("El nombre debe contener solo letras")
        self._nombre = valor.capitalize()

    @property
    def apellido(self):
        return self._apellido

    @apellido.setter
    def apellido(self, valor):
        if not valor.isalpha():
            raise ValueError("El apellido debe contener solo letras")
        self._apellido = valor.capitalize()

    @property
    def sexo(self):
        return self._sexo
  
```

The interface includes a sidebar with a project tree, a top toolbar with icons for running and debugging, and a status bar at the bottom indicating the file path and encoding.

This screenshot shows the continuation of the `Persona` class from the previous image. The code includes:

```

    @sexo.setter
    def sexo(self, valor):
        if valor.lower() not in ["masculino", "femenino", "prefiero no decirlo"]:
            raise ValueError("Sexo debe ser 'Masculino', 'Femenino' o 'Prefiero no decirlo'")
        self._sexo = valor.lower()

    @property
    def email(self):
  
```

The IDE interface remains consistent, showing the same project structure and toolbars.

The screenshot shows a code editor with a project named 'PROYECTOSMB' and a file named 'PERSONA.py'. The code defines a class 'Persona' with the following methods and attributes:

```

class Persona:
    def __init__(self, nombre, apellido, cedula, sexo, email):
        self._nombre = nombre
        self._apellido = apellido
        self._cedula = cedula
        self._sexo = sexo
        self._email = email

    @property
    def nombre(self):
        return self._nombre

    @property
    def apellido(self):
        return self._apellido

    @property
    def cedula(self):
        return self._cedula

    @property
    def sexo(self):
        return self._sexo

    @property
    def email(self):
        return self._email

    @email.setter
    def email(self, valor):
        if valor is None:
            self._email = None
        else:
            patron = r'^[a-zA-Z0-9_+&quot;@]{1,64}$'
            if not re.match(patron, valor):
                raise ValueError("Email inválido, digite correctamente")
            self._email = valor

    # Representación en cadena
    def __str__(self):
        return f'Persona: {self.nombre} {self.apellido}, Cédula: {self.cedula}, Sexo: {self.sexo}, Email: {self.email}'
  
```

The status bar at the bottom indicates the file is 'PERSONA.py' in the 'DOMINIO' folder, using Python 3.13 (PROYECTOSMB) with CRLF line endings, UTF-8 encoding, and 4 spaces for indentation.

The screenshot shows a code editor with a project named 'PROYECTOSMB' and a file named 'PERSONA.py'. The code defines a class 'PersonaServicio' that inherits from 'QMainWindow'. The class implements the following methods:

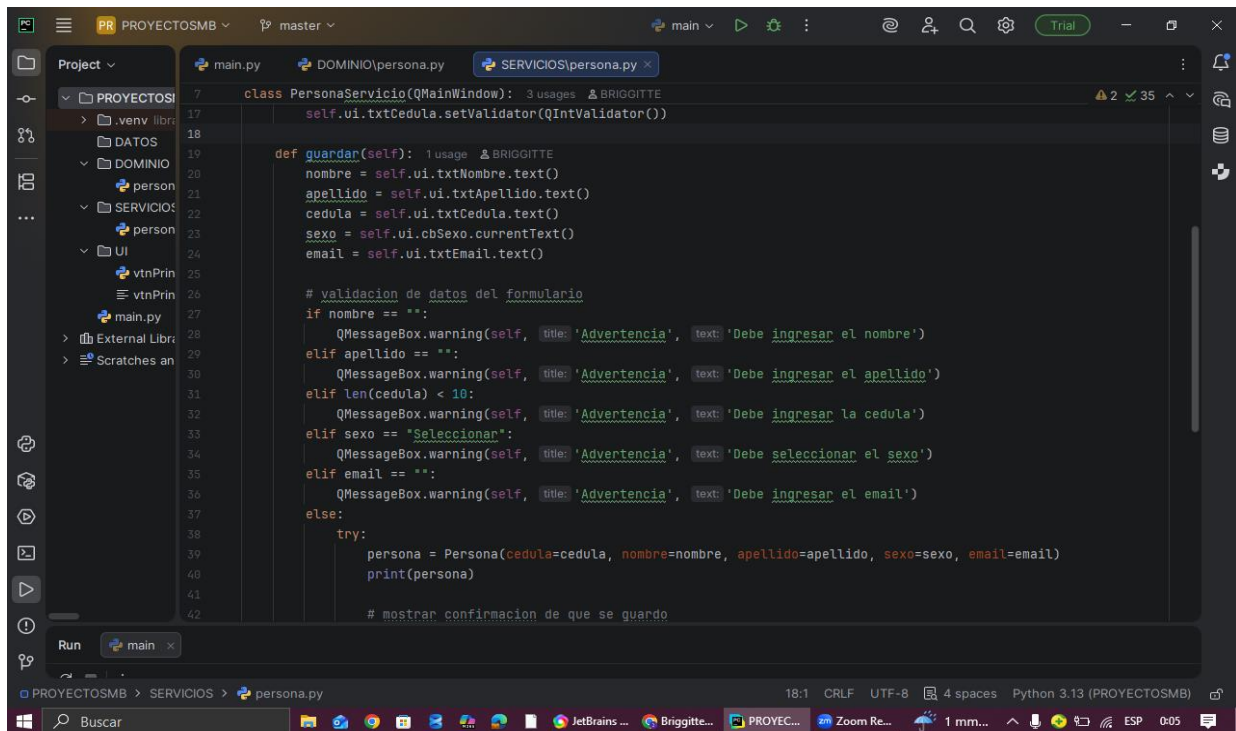
```

class PersonaServicio(QMainWindow):
    def __init__(self):
        super(PersonaServicio, self).__init__()
        self.ui = Ui_vtnPrincipal()
        self.ui.setupUi(self)
        self.ui.btGuardar.clicked.connect(self.guardar)
        self.ui.btLimpiar.clicked.connect(self.limpiar)
        self.ui.txtCedula.setValidator(QIntValidator())

    def guardar(self):
        nombre = self.ui.txtNombre.text()
        apellido = self.ui.txtApellido.text()
        cedula = self.ui.txtCedula.text()
        sexo = self.ui.cbSexo.currentText()
        email = self.ui.txtEmail.text()

        # validacion de datos del formulario
        if nombre == "":
  
```

The status bar at the bottom indicates the file is 'PERSONA.py' in the 'SERVICIOS' folder, using Python 3.13 (PROYECTOSMB) with CRLF line endings, UTF-8 encoding, and 4 spaces for indentation.



```

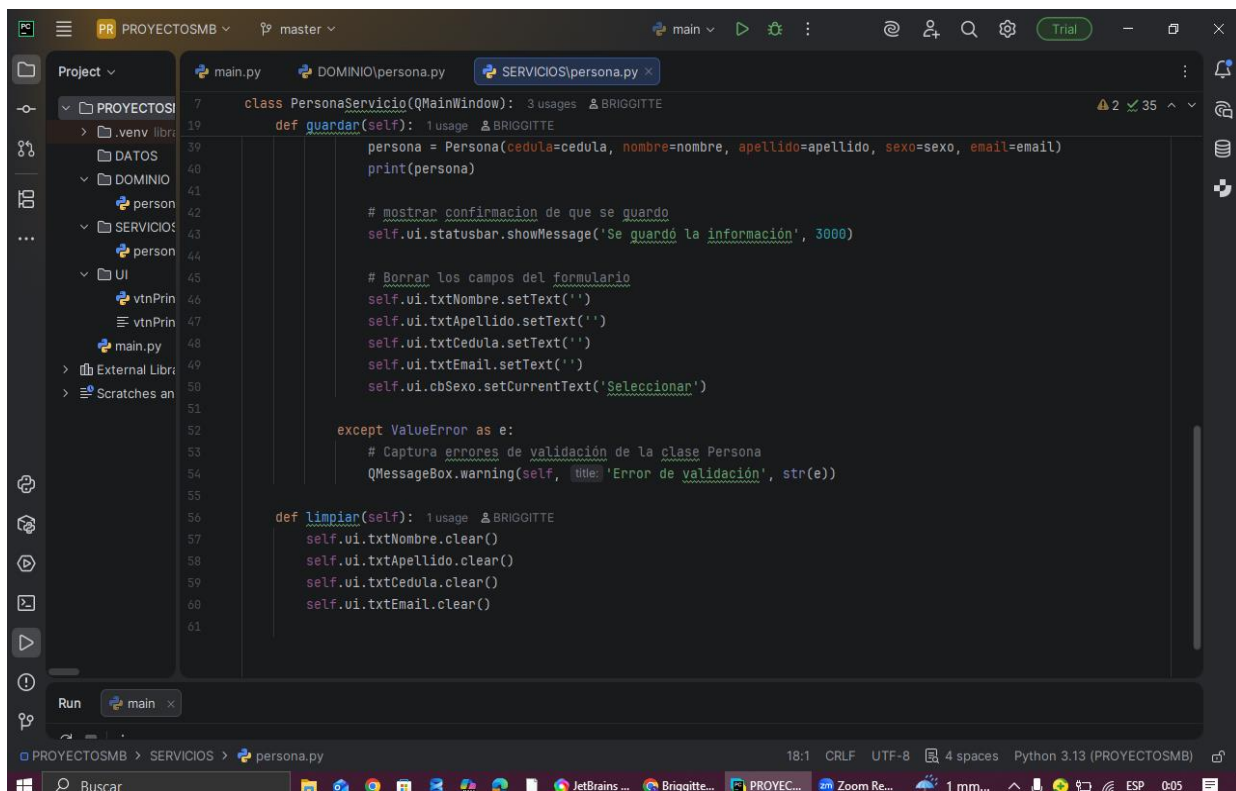
class PersonaServicio(QMainWindow):
    def __init__(self):
        self.ui.txtCedula.setValidator(QIntValidator())

    def guardar(self):
        nombre = self.ui.txtNombre.text()
        apellido = self.ui.txtApellido.text()
        cedula = self.ui.txtCedula.text()
        sexo = self.ui.cbSexo.currentText()
        email = self.ui.txtEmail.text()

        # validacion de datos del formulario
        if nombre == "":
            QMessageBox.warning(self, 'Advertencia', text: 'Debe ingresar el nombre')
        elif apellido == "":
            QMessageBox.warning(self, 'Advertencia', text: 'Debe ingresar el apellido')
        elif len(cedula) < 10:
            QMessageBox.warning(self, 'Advertencia', text: 'Debe ingresar la cedula')
        elif sexo == "Seleccionar":
            QMessageBox.warning(self, 'Advertencia', text: 'Debe seleccionar el sexo')
        elif email == "":
            QMessageBox.warning(self, 'Advertencia', text: 'Debe ingresar el email')
        else:
            try:
                persona = Persona(cedula=cedula, nombre=nombre, apellido=apellido, sexo=sexo, email=email)
                print(persona)

                # mostrar confirmacion de que se guardo

```



```

                persona = Persona(cedula=cedula, nombre=nombre, apellido=apellido, sexo=sexo, email=email)
                print(persona)

                # mostrar confirmacion de que se guardo
                self.ui.statusbar.showMessage('Se guardó la información', 3000)

                # Borrar los campos del formulario
                self.ui.txtNombre.setText('')
                self.ui.txtApellido.setText('')
                self.ui.txtCedula.setText('')
                self.ui.txtEmail.setText('')
                self.ui.cbSexo.setCurrentText('Seleccionar')

            except ValueError as e:
                # Captura errores de validación de la clase Persona
                QMessageBox.warning(self, 'Error de validación', str(e))

    def limpiar(self):
        self.ui.txtNombre.clear()
        self.ui.txtApellido.clear()
        self.ui.txtCedula.clear()
        self.ui.txtEmail.clear()

```